DOS GAME REVERSE ENGINEER PROJECT

1 Introduction

Reverse engineering is the process of analyzing a software system to understand its internal structure, behavior, and functionality without access to its original source code. This technique is commonly used in security research, debugging, and software analysis to identify how programs operate at a low level, including how data is stored, processed, and modified during execution.

Legacy games, particularly those developed between 1990 and 2005, are often vulnerable to reverse engineering and runtime manipulation due to the absence of modern security mechanisms. These systems typically lack features such as memory protection, code obfuscation, anti-debugging techniques, and integrity verification. As a result, critical game variables like player health, score, and ammunition are stored in plain memory and can be easily accessed and modified using external tools such as debuggers and memory scanners.

The purpose of this experiment is to apply reverse engineering techniques to analyze and manipulate the runtime behavior of a legacy game. Specifically, the study focuses on identifying how the player's health value is stored and updated in memory during gameplay. By using tools such as Cheat Engine and a debugger, the relevant assembly instructions responsible for modifying the health value are located and altered. The final objective is to modify the program's behavior so that the player's health does not decrease, demonstrating how insecure software design can be exploited and highlighting the importance of secure development practices.

2 Game Selection & Environment Setup

2.1 Game Details

The selected game for this experiment is **DOOM**, a classic first-person shooter released in the early 1990s. DOOM was chosen because it falls within the required legacy software period (1990–2005) and lacks modern security mechanisms such as memory protection and anti-debugging techniques. This makes it a suitable candidate for reverse engineering and runtime manipulation.

The game was executed using **DOSBox-X**, an advanced DOS emulator that provides debugging capabilities. DOSBox-X allows the game to run in a controlled environment while enabling low-level inspection of memory and assembly instructions during execution.

2.2 Tools Used

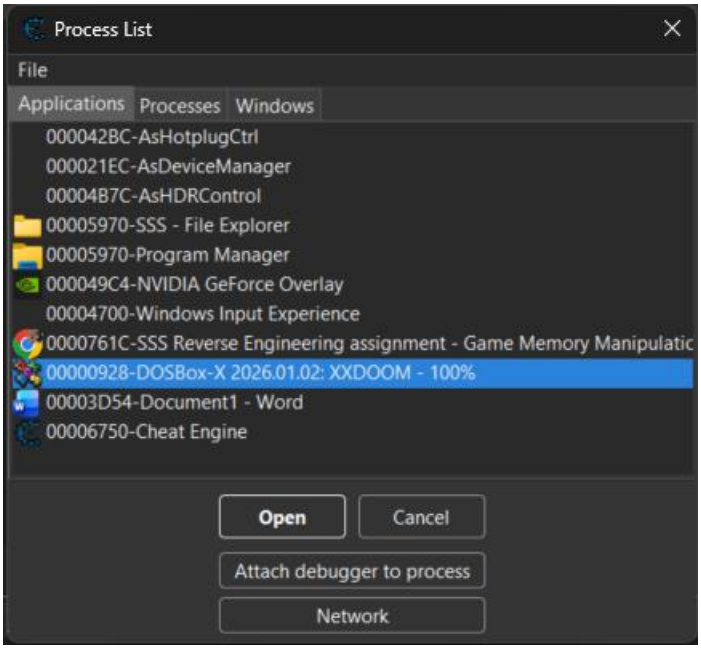
The following tools were used to perform the analysis:

- **Cheat Engine**
  Cheat Engine was used for dynamic memory analysis. It allowed scanning of memory to identify the player's health value, filtering multiple results, and monitoring how values change during gameplay. It was also used to track which instructions read from or write to specific memory addresses.

- **DOSBox-X Debugger**
  The built-in debugger in DOSBox-X was used to analyze the game at the assembly level. It enabled viewing of disassembled instructions, setting breakpoints, and stepping through execution to understand how the game updates the player's health during runtime.

3 Methodology

3.1 Attaching the DOSBox to cheat engine



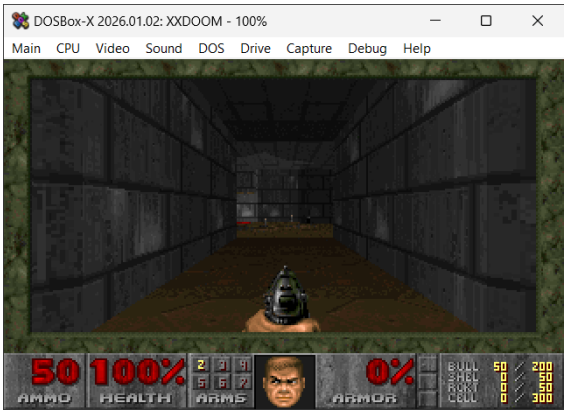
3.2 Initial Memory Scan

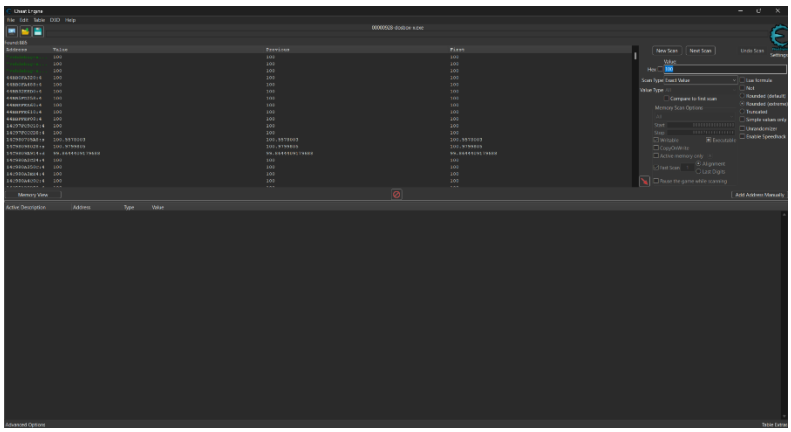
The analysis began by launching the game DOOM within the DOSBox-X environment and attaching Cheat Engine to the running process. The player's initial health value (e.g., 100) was observed from the game interface.

An initial memory scan was performed in Cheat Engine using the following configuration:

- Value Type: **All**
- Scan Type: **Exact Value**
- Value: **Current health (e.g., 100)**

This scan returned a large number of memory addresses that potentially contained the health value.



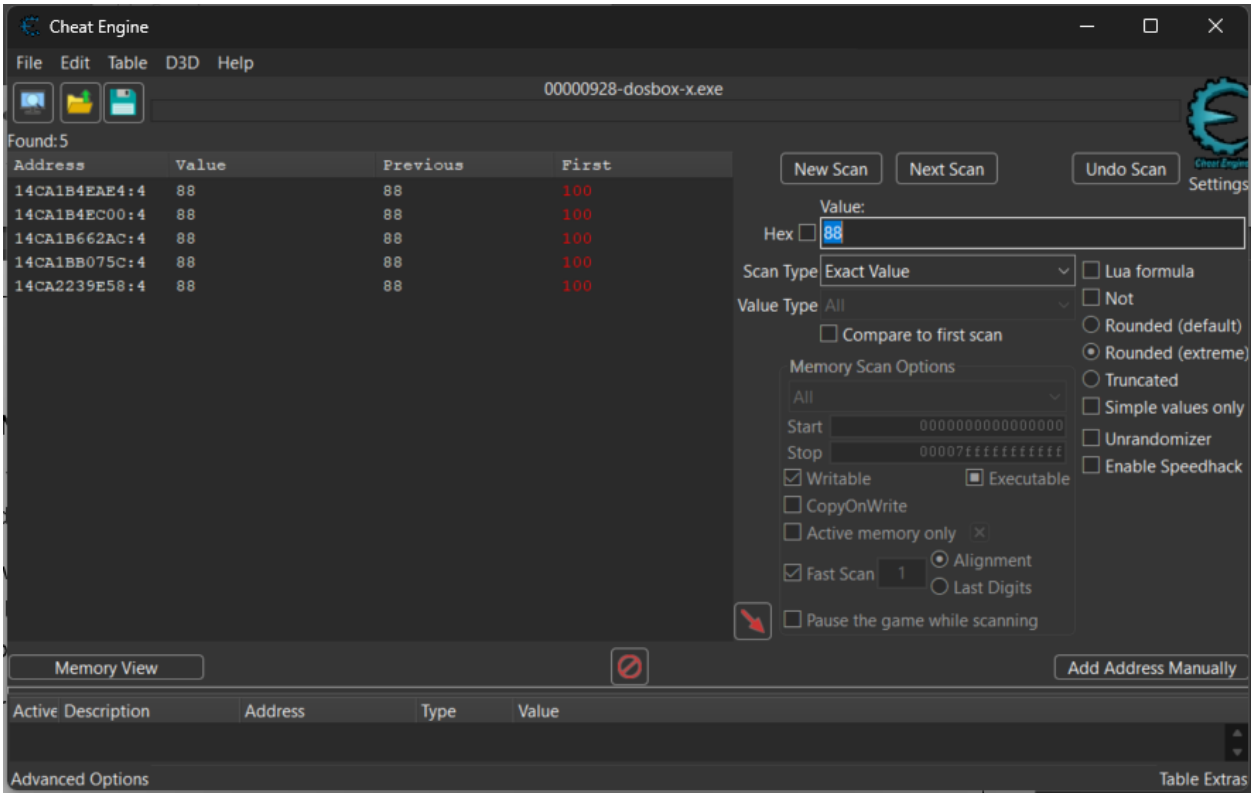


3.3 Filtering Memory Values

To narrow down the results, the player was intentionally damaged in the game, causing the health value to decrease (e.g., from 100 to 90).

A second scan was then performed using the updated health value. This process was repeated multiple times (taking damage and rescanning), progressively filtering out irrelevant memory addresses.

After several iterations, the number of candidate addresses was reduced significantly.



3.4 Identification of Multiple Health Addresses

After sufficient filtering, **five memory addresses** were identified that appeared to hold the player's health value. These addresses showed identical or closely related values.

When modifying the value of one of these addresses (e.g., setting it to 200), it was observed that:

- Four addresses changed simultaneously
- The displayed health value in the game interface also changed

This indicated that these addresses were **not independent**, but rather **linked or synchronized copies** of the same value.

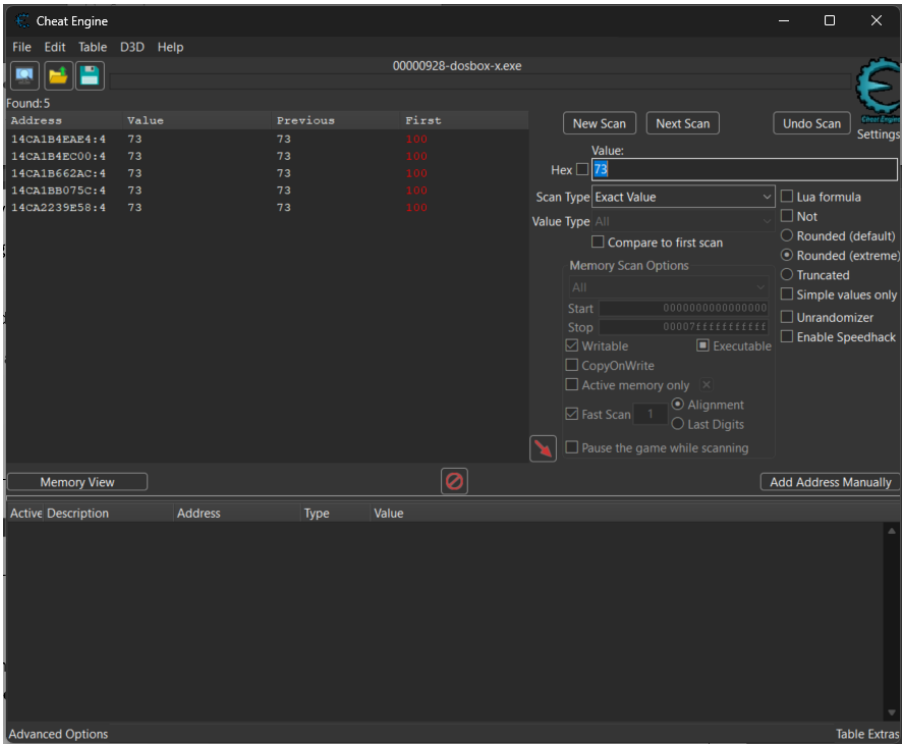


Figure 1List of the five identified addresses in Cheat Engine

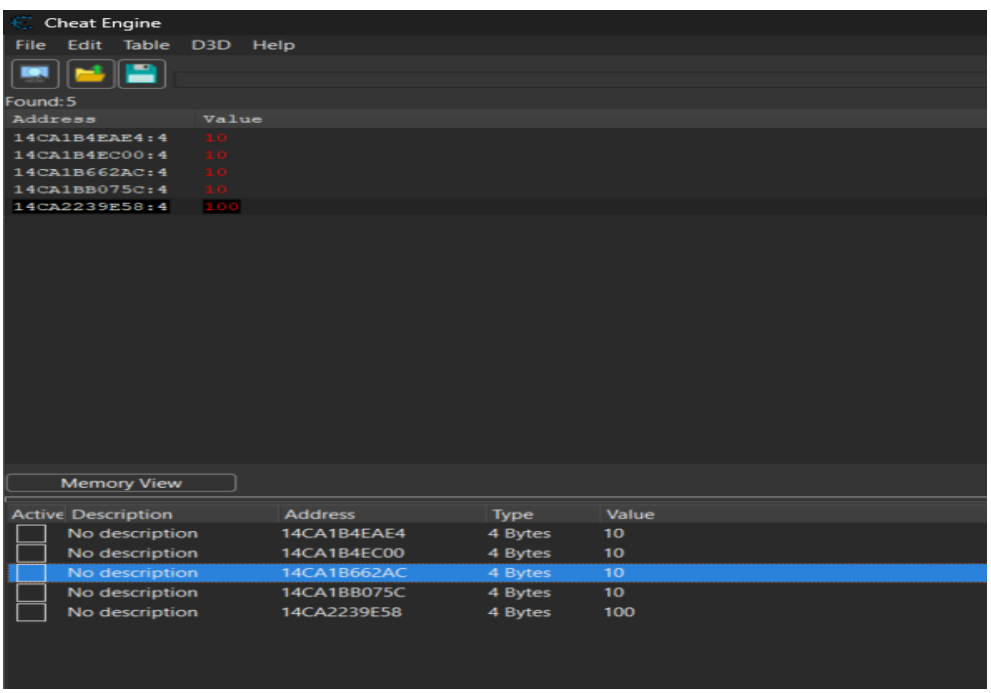


Figure 2Demonstration of modifying one address and four values changing

3.5 Differentiating Front-End and Back-End Values

Further testing revealed an important distinction between **display (front-end)** values and the **actual game logic (back-end)** value.

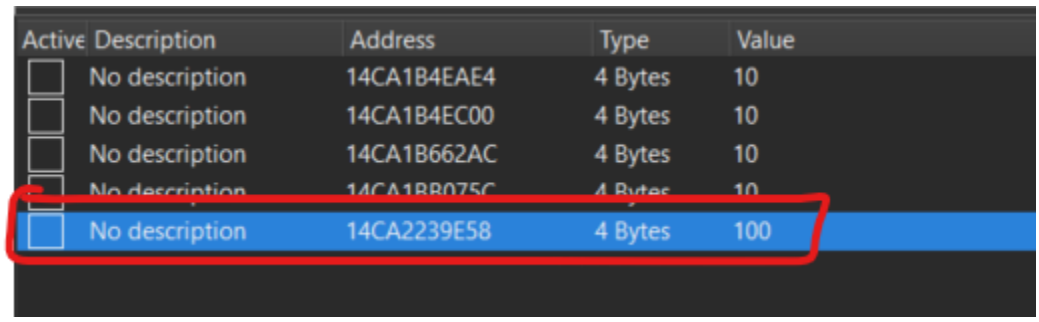
- Modifying the previously identified five addresses changed the **displayed health value** in the game
- However, when the health reached zero during gameplay, the player still died, even if the displayed value was artificially increased

This behavior indicated that these four addresses were **front-end or display-related values**, used for rendering the health on the screen rather than controlling the actual game logic.

To identify the true health value, additional scanning and testing were performed. A separate memory address was discovered that behaved differently:

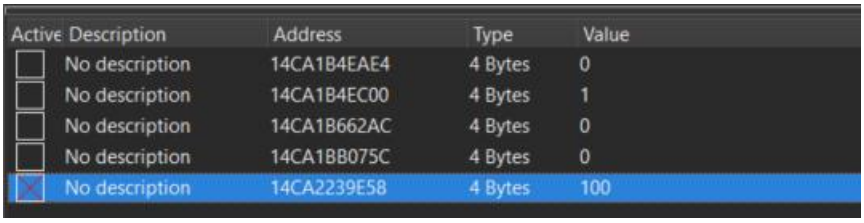
- Changing this value did **not affect the health displayed immediately**
- However, freezing this value prevented the player from dying, even when the visible health reached zero

This confirmed that this address represented the **real back-end health value** used by the game's internal logic.



Active	Description	Address	Type	Value
	No description	14CA1B4EAE4	4 Bytes	10
	No description	14CA1B4EC00	4 Bytes	10
	No description	14CA1B662AC	4 Bytes	10
	No description	14CA1BB075C	4 Bytes	10
	No description	14CA2239E58	4 Bytes	100

Figure 3Address identified as real health value in Cheat Engine



Active	Description	Address	Type	Value
	No description	14CA1B4EAE4	4 Bytes	0
	No description	14CA1B4EC00	4 Bytes	1
	No description	14CA1B662AC	4 Bytes	0
	No description	14CA1BB075C	4 Bytes	0
	No description	14CA2239E58	4 Bytes	100

Figure 4 Freez the address operation

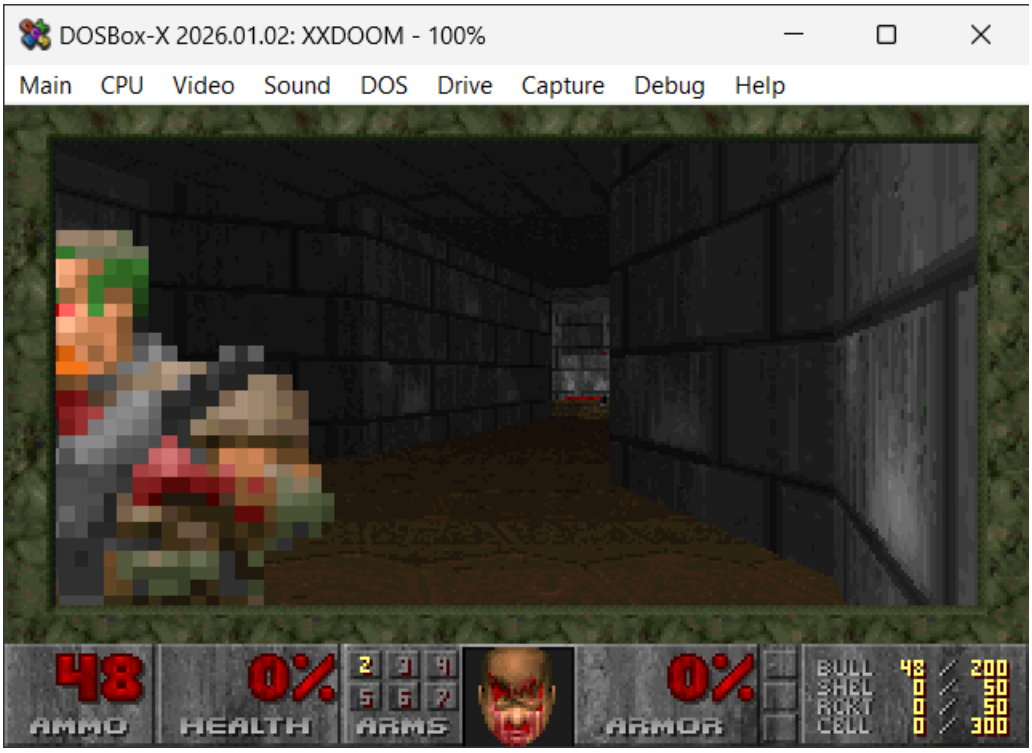


Figure 5 Health got zero but character still alive

3.6 Tracking Memory Access Using Debugger

Once the real health address was identified, Cheat Engine's debugging feature was used to analyze how the value is modified during gameplay.

The option **"Find out what writes to this address"** was selected for the identified back-end health address. This allowed monitoring of assembly instructions that update the health value in memory.

When the player received damage (e.g., getting shot), the debugger captured the instruction responsible for modifying the health value.

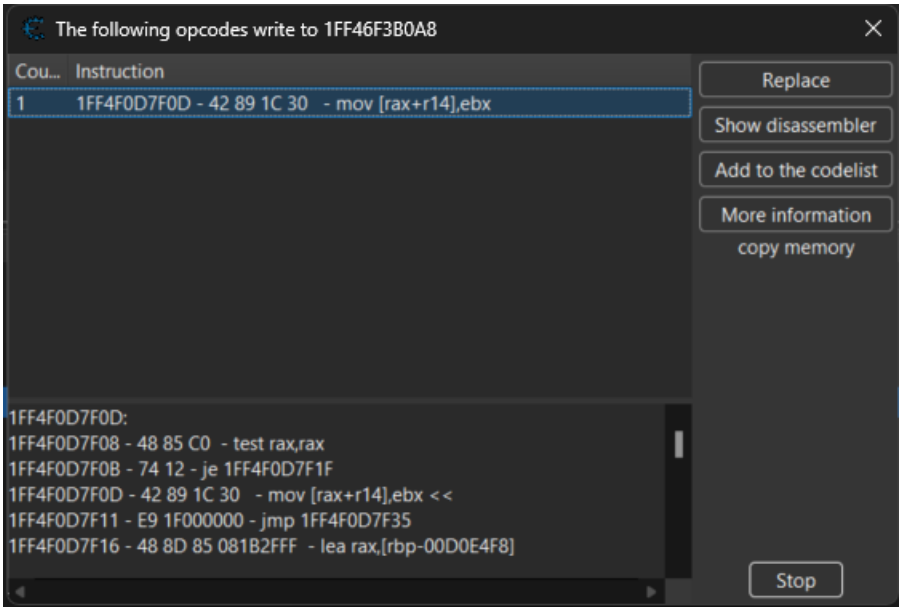


Figure 6 Find out what writes to this address

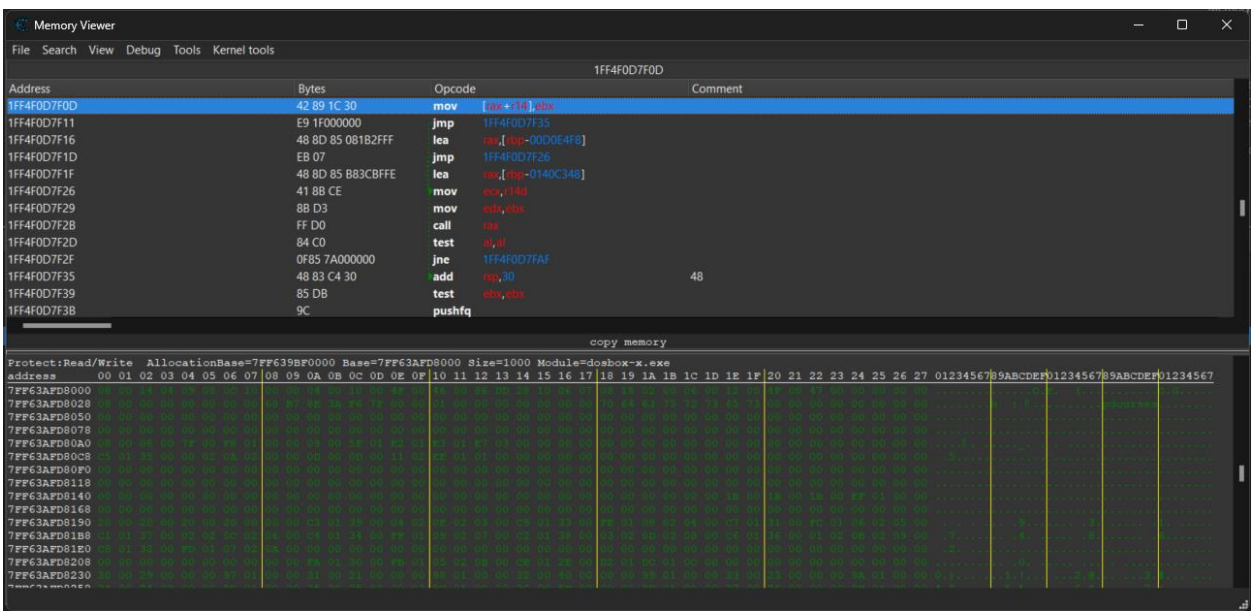


Figure 7Debugger window showing instructions writing to the health address

3.7 Identifying the Damage Instruction

By analyzing the captured instructions and setting breakpoints, the following instruction was identified:

sub ebx, edx

This instruction was triggered exactly when the player took damage.

Through register analysis:

- **EBX** was identified as holding the current health value
- **EDX** was identified as holding the damage value

Thus, the instruction performs the operation:

health = health - damage

This confirmed that it is the **core instruction responsible for decreasing player health**.

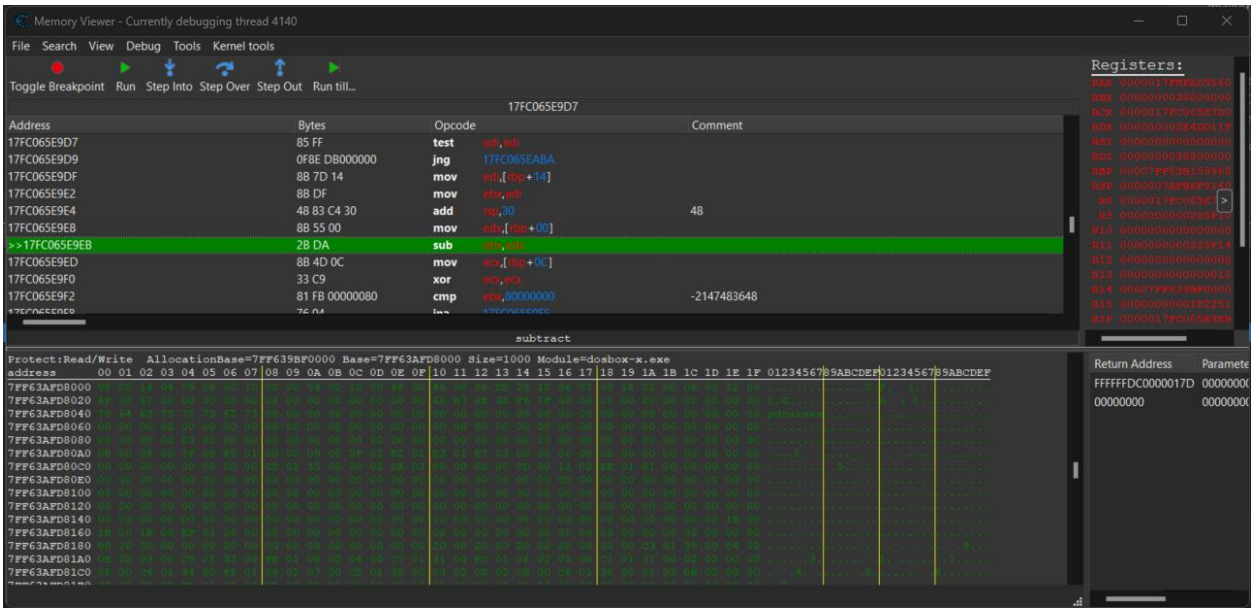


Figure 8 Breakpoint hit on sub ebx, edx when player takes damage

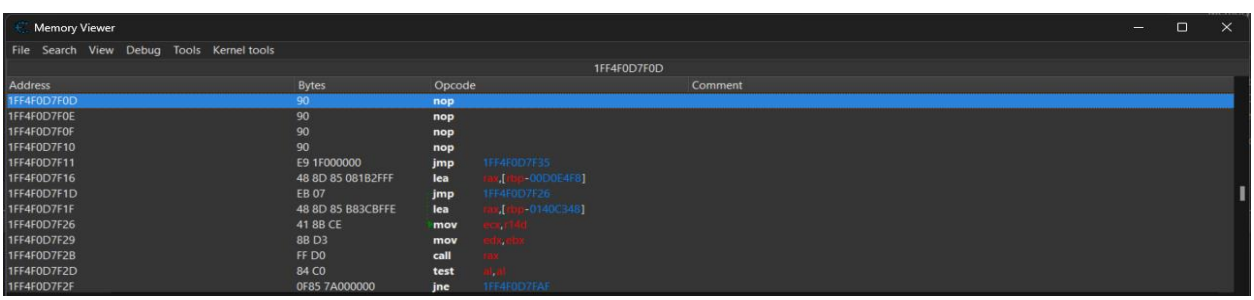


Figure 9 Replace the instructions with nop

3.8 Summary of Findings

Through the methodology, the following key observations were made:

- Multiple memory addresses initially identified were **front-end (display) values**
- These values were synchronized and did not control the actual game logic
- A separate memory address was identified as the **true back-end health value**
- The instruction sub ebx, edx was confirmed as the **primary operation responsible for health reduction**

4 Security Analysis & Discussion

4.1 Why the Attack is Possible

The successful manipulation of the player's health value demonstrates that the game lacks fundamental security mechanisms that are commonly present in modern software systems. Several factors contribute to the feasibility of this attack.

- **Absence of Memory Protection**

The game stores critical variables such as player health directly in memory without any protection mechanisms. This allows external tools like Cheat Engine to scan, locate, and modify these values during runtime.

In modern systems, memory protection techniques such as access control, address space layout randomization (ASLR), and protected memory regions are used to restrict unauthorized access. However, in this legacy game, memory is freely readable and writable, making it highly vulnerable to manipulation.

- **Lack of Encryption**

All in-game values, including health, are stored in plain (unencrypted) form in memory. This makes it easy to identify meaningful values during memory scanning, especially when the value is known (e.g., health = 100).

If encryption or encoding techniques were used, the stored value would not directly match the visible value, making it significantly more difficult to locate and manipulate.

- **No Anti-Debugging Mechanisms**

The game does not include any anti-debugging techniques to detect or prevent the use of debugging tools. As a result, it was possible to:

- Attach a debugger to the running process
- Monitor memory access
- Set breakpoints on instructions
- Step through assembly code

This allowed precise identification of the instruction responsible for reducing health (sub ebx, edx).

Modern secure applications often implement anti-debugging techniques such as debugger detection, execution timing checks, and code obfuscation to prevent such analysis.

4.2 Identified Security Weaknesses

Based on the analysis, several critical weaknesses in the game's design were identified.

- **Plain Memory Storage of Critical Data**

The player's health value is stored directly in memory in a readable and modifiable format. This makes it easy to locate using memory scanning techniques.

Additionally, multiple copies of the health value (front-end/display values) were found, indicating that the game does not securely manage or isolate critical data.

- **Lack of Input or Value Validation**

The game does not validate changes made to the health value during runtime. When the value was modified externally (e.g., setting health to 200), the game accepted the change without verification.

This lack of validation allows attackers to inject arbitrary values into critical variables without restriction.

- **Absence of Integrity Checks**

There are no mechanisms to verify whether the game code or memory has been altered. For example:

  - No checksum or hash validation of instructions
  - No verification of memory consistency
  - No detection of modified assembly instructions (e.g., NOP replacement)

As a result, it was possible to modify the instruction sub ebx, edx without triggering any protection or error.

4.3 Proposed Mitigation Techniques

To prevent such attacks in modern software systems, several security techniques can be implemented.

- **Code Obfuscation**

Code obfuscation transforms the program into a form that is difficult to analyze while preserving its functionality. This includes:

- Renaming variables and functions
- Inserting misleading instructions
- Flattening control flow

Obfuscation makes it significantly harder for attackers to understand and locate critical logic such as health calculations.

- **Anti-Debugging Techniques**

Modern applications can include mechanisms to detect and respond to debugging attempts. Examples include:

- Checking for attached debuggers
- Detecting breakpoints or modified instructions
- Monitoring execution timing anomalies

If a debugger is detected, the program can terminate or alter its behavior to prevent analysis.

- **Integrity Checks (Hashing)**

Integrity verification ensures that the program code and memory have not been tampered with. This can be implemented using:

- Hashing critical sections of code
- Verifying checksums at runtime
- Detecting unauthorized modifications

If any changes are detected (e.g., NOP injection), the program can trigger an error or shut down.

- **Server-Side Validation**

In modern applications, critical logic is often handled on a server rather than the client. This prevents users from modifying important values locally.

For example:

- Health calculations can be validated on a server
- Any abnormal values can be rejected

This approach is commonly used in online games to prevent cheating.

- **Memory Protection Mechanisms**

Advanced memory protection techniques can restrict access to sensitive data. These include:

- Marking memory regions as read-only or non-executable
- Using pointer encryption or address randomization
- Isolating critical variables in protected memory spaces

Such techniques make it significantly harder to locate and modify important variables like health.

4.4 Summary

The experiment demonstrates that the lack of basic security mechanisms in legacy software makes it highly vulnerable to reverse engineering and runtime manipulation. By exploiting these weaknesses, it was possible to identify and modify critical game logic at the assembly level.

This highlights the importance of incorporating security measures such as obfuscation, integrity checks, and memory protection in modern software development to prevent unauthorized manipulation.